

## Título

### IR-Board: Requirements management platform

## Resumen

Se pretende crear una herramienta que permita dar soporte a los procesos involucrados en la gestión de requisitos: gestión de fuentes e interesados, obtención y refinado de requisitos permitiendo la comprobación de los atributos de calidad, y dando soporte a distintas estándares y normas (guías del IEEE 830 y el estándar de la ISO 29148) Se incorporarán facilidades para incluir información de técnicas estáticas y dinámicas (como casos de uso, diagramas de flujo y tablas de decisión) y la aplicación de metodologías ágiles como las historias de usuario.

## Decisiones arquitectónicas

- Se decidió un sistema web sobre standalone debido a la facilidad de escalado, así como una mayor predisposición al trabajo colaborativo, natural en la administración requisitos.

Se elige React con Material UI debido a su conocimiento previo, además de una gran versatilidad y enfoque destinado a portabilidad y adaptabilidad. Se prefiere sobre la madurez de otros frameworks como Angular, o sobre bibliotecas de componentes como Ant Design o TailwindCss debido a dicho enfoque y simplificación del desarrollo.

- Se elige SpringBoot sobre Express.js por el entorno más seguro por defecto, y valorando que ambos tienen una buena integración con la tecnología de frontend elegida.
- A fin de facilitar su implantación en otros sistemas, se decide separar el frontend y backend en proyectos separados para facilitar su adaptación a otros entornos.
- Arquitectura web simulando trabajo colaborativo similar al word con los mutex a nivel de párrafo = requisito
- La asociación entre elementos a fin de marcar como pendientes de revisar debe ser bidireccional, y de forma que el patrón observer debe estar unido para cada entidad. Para evitar bucles infinitos, se puede hacer una revisión por separado de una modificación, que a efectos, aplica el mutex de la misma manera, y o bien valida o modifica aquello marcado. Se ha decidido ante la visión que, aunque se pueda entrar en un bucle de modificar todo, forma parte del proceso de refinado, y es en esencia, una buena práctica.
- Arquitectura de microservicios – Traefik - Gateway (ya tengo una mínima experiencia con este de ASR) – React - frontend – Ory Kratos - Identification & Sessions – Ory Oathkeeper - Enforcement – Ory Keto - Manages complex – springboot - backend con el RMS – PostgreSQL - BD Identidad para Ory Kratos – PostgreSQL - BD de tuplas para Ory Keto – PostgreSQL - BD del RMS – Loki - Logging centralizado – Promtail - Agente que recolecta los logs para Loki – Grafana - Dashboard para métricas y logs de Loki
- Consideraciones de la arquitectura a indicar que valoré pero descarté  
Ory Hydra - (unneded because I'm not an identity provider) Ory Keto - Authorization (unneded because the role groups are simple) gateways Kong y Typhoon.
- Elegí microservicios por una mayor escalabilidad e integración más sencilla con servicios existentes.
- Elegí hacer uso de sistemas ya probados de seguridad modulares, como Ory, así como la gateway de traefik, debido a la complejidad, trabajo añadido y probablemente menor eficiencia que tendría

crearlos por mi cuenta. Asimismo, evitarlo me asegura un proyecto más sólido, con más facilidad de añadir nuevas funcionalidades, más seguro, y más eficiente.

- Elección de postgres como base de datos relacional debido a que el dominio es estable, se trata de requisitos y datos a guardar considerados estándar, y debido a su integración preferida por Ory. Además, permite usar columnas JSONB, que ofrecen la flexibilidad de bases de datos NoSQL para tener atributos personalizados. Por último, tiene una buena integración con un driver de mucha madurez con springboot.
- Elegí separar en dos redes la arquitectura para dar una mayor seguridad a la arquitectura siguiendo el principio de menor privilegio.
- 2026/01/31 I decided that when flagging an element, it will have a `is_suspect` boolean value, and a `impact_summary` field. Whenever a new flagging occurs, it will simply check the flag and set it if it's false, and append why to the `impact_summary`. Reviewing it removes everything in its `impact_summary` field.
- 2026/01/31 I decided that to ensure natural ordering for a better UX, the identifiers are calculated on the frontend virtually. To ensure a way to order, the entities will have both a UUID and a `sort_order` field, with float values. If an element were to be placed between element 1 and 2, its `sort_order` would be its midpoint: 1.5. To ensure a problem would not happen, if the difference between two numbers is less than a constant, a re-normalization would occur, iterating over all elements and setting their `sort_order` to their integer index.
- 2026/02/02 I decided that the project, before a baseline is saved, will allow reordering of requirements, which will make requirements have a dynamic semantic ID (FR-UM-1.5 -> functional requirement user management 1.5). This ensures a better UX until it is reasonable that the project is past its inception, and then will lock the semantic IDs once the first baseline is set. Any semantic ID used before, even if deactivated, won't be used again, ensuring compliance with ISO 29148 and its traceability.
- 2026/02/02 I decided that in order to comply with ISO 29148, the system can't allow any user to delete anything, but rather deactivate it.
- 2026/02/05 Along with my tutors, we decided to revoke decision 2026/02/02 always allow reordering of entities, with the corresponding modification of the entity's dynamic id. This was done to prioritize UX. To comply with ISO 29148's traceability, it will be implemented via the internal UUID; To ensure users can follow this traceability, the system will show both the dynamically generated id and the internal UUID of the entities, and the system will allow to search by UUID.
- 2026/02/05 Along with my tutors, we decided to add a set of states for all entities of a project, as suggested by the lifecycle of an element on ISO 29148. The flagging as pending a review will be absorbed into this system.
- 2026/02/05 Along with my tutors, we decided that for now, all documents linked to a project are global and accessible by those linked to said project. Access control for these elements falls beyond the scope of this project.
- 2026/02/05 Along with my tutors, we decided that the linking will be first via a context menu, and if the project advances at a sufficiently rapid pace, implement a markdown style linking. The linking between elements will be unidirectional after all, as linking a requirement to another does not imply that the observed requirement is affected by the observer, and similarly with linking to a

stakeholder or to a document, the recipient of the observation is most of the times, unlikely to change, and when it does is when one would be interested in being notified.

- 2026/02/06 requirement engineer will have access via linking with read or write access to each functionality on a project, not level access. This is done to avoid breaking the workflow due to a req. engineer needing to modify a parent and not being able to do so.

–2026/02/06 las dependencias a usar en el backend de springboot son Lombok, Spring Web, Spring Boot DevTools, H2 Database, Spring Data JPA, PostgreSQL Driver, y Validation.

## Requisitos no funcionales

- RN-01 Persistencia Normativa: La base de datos debe permitir la trazabilidad exigida por la ISO 29148 (id, descripción, prioridad, estado, origen, justificación).
- RN-02 Control de Concurrencia: El sistema debe implementar un mecanismo de bloqueo a nivel de requisito (mutex) para evitar condiciones de carrera durante la edición simultánea.
- RN-03 Disponibilidad y Web: Acceso multiplataforma mediante navegador, garantizando un diseño responsive gracias a Material UI.

## Future Work / Enhancements

### Document management

- (future work) The system must allow a requirement engineer fill a tabular use case
  - The system must have the following fields:
    - Name
    - Description
    - Actors
    - Initial Condition
    - Preconditions
    - Postconditions
    - Normal flow
    - Alternative flows
    - Exceptions
- (future work) The system must allow a requirement engineer fill a scenario
  - The system must have the following fields:
    - Name
    - Description
    - Actors
    - Initial Condition
    - Preconditions
    - Postconditions
    - Flow
- The system must allow a project manager generate a customized pdf of requirements for a stakeholder
- The system must allow a project manager define a stencil for pdf generation of the srs export

### Variant control

Highly ambitious—only if time permits. Keep this in mind even if not implemented: requirement reuse. Defining templates or abstract projects. In the industry, “copy and paste” is the standard approach.

### **Search and filtering**

The system must allow a user to search requirements by text

### **extras**

in of frontend & databases with AI smtp server